

Домашнее задание 11

Лиза Фоменко

Задание 1

Рассмотрим алгоритм Дейкстры с модификацией, в которой он сохраняет для каждой вершины предыдущую на кратчайшем пути в массиве предков $\pi[v]$. Докажите, что граф, состоящий из вершин исходного и ребер вида $\pi[v] \rightarrow v$, является деревом.

Дерево - это связный ациклический граф.

Некоторые замечания: 1) если существует вершина, не достижимая из вершины-начала алгоритма, то для нее нет значения в π , так как расстояния не существует. Согласно Вашей постановке задачи такая вершина будет просто болтаться в графе, то есть он связным не будет. 2) опять же судя по Вашей постановке непонятно, как мы проводим ребра $\pi[v] \rightarrow v$. Для доказательства дерева эти ребра очевидно должны быть направленными, так как иначе мы получаем DAG.

Поэтому для доказательства прописываю, что 1) в графе нет вершин, недостижимых из вершины-начала алгоритма; 2) итоговый граф по построению неориентированный

Тогда нам нужно доказать, что граф, который мы получим, 1) связный; 2) ациклический

Докажем

1) что граф связный, то есть существует путь от любой вершины до любой другой.

по построению ребер $\pi[v] \rightarrow v$ мы соединяем вершину с ее предшественником в минимальном пути $s \rightarrow v$. Алгоритм Дейкстры находит минимальный путь от s до любой (достижимой) вершины v , то есть существует путь из s до любой вершины графа. Все эти ребра входят в $\pi[v]$ по построению. Так как граф мы строим неориентированным, то также существует путь из любой вершины v в s , из этих фактов следует, что существует путь $v \rightarrow u, u \rightarrow v$ для любой пары вершин.

2) граф ациклический. Докажем от противного, пусть в нашем получившемся графе получился цикл, то есть пускай есть цикл $v_i, \dots, v_k, v_i$. Это значит, что: $\pi[v_{i+1}] = v_i, \dots, \pi[v_k] = v_i$. Пусть существует путь $s \rightarrow v_i$, тогда по алгоритму Дейкстры и правилу релаксации: $d(s, v_k) \geq d(s, v_{k-1}) \geq d(s, v_{i+1}) \geq d(s, v_i)$. Тогда, по построению π , мы релаксировали ребро $v_k \rightarrow v_i$, тогда верно, что $d(s, v_i) = d(s, v_k) + d(v_k, v_i)$. Но ведь мы уже доказали, что $d(s, v_k) \geq d(s, v_{k-1}) \geq d(s, v_{i+1}) \geq d(s, v_i)$, а веса всех ребер положительны, значит эти условия не могут быть выполнены одновременно. Значит, в графе не может быть циклов.

Тогда мы получили, что полученный граф - связный и ациклический, то есть является деревом.

Задание 2

Предложите $O(|V|+|E|)$ алгоритм поиска кратчайших расстояний от данной вершины s до всех остальных в графе с весами ребер 0 или 1. Докажите его корректность и оцените асимптотику.

Насколько я помню, алгоритм называется 0-1 BFS. он создан специально для таких графов, с ребрами весом только 0 и 1, и, как указано в названии, работает через BFS за $O(|V| + |E|)$ соответственно.

Он работает на основе структуры deque.

```
def 01_BFS(graph, dist, start_v):
```

```
    dist[start_v] = 0          # от start до start 0
```

```
    dq = deque()              # это deque
```

```
    # кладем стартовую вершину в начало очереди
```

```
    dq.appendleft(vertex_source)
```

```
    # пока в deque находятся к-л вершины
```

```
    while dq:
```

```
        node = dq.popleft()    # достаем первую в очереди вершину
```

```

# проверяем на релаксацию
for edge in graph[node]:
    childNode, weight = edge    # сосед + вес ребра

    # аналог Дейкстры - релаксация, если нужно
    if dist[childNode] > dist[node] + weight:
        dist[childNode] = dist[node] + weight

    # если вес ребра = 1 (максимальный из возможных), то кладем В КОНЕЦ очереди
    if weight:
        dq.append(childNode)

    # если вес ребра = 0 (минимальный из возможных), то кладем в НАЧАЛО очереди
    else:
        dq.appendleft(childNode)

```

Когда мы делаем обычный BFS, то обходим вершины по уровням, то есть в любой момент времени у нас в очереди находятся вершины либо только одного уровня, либо только двух соседних слоев. В случае dequeue в очереди одновременно находятся только элементы с двумя разными значениями `current_weight`.

В нашем случае мы используем deque (как и в обычной очереди добавление/вытаскивание элемента за $O(1)$). Рассмотрим начальную вершину. Рассмотрим всех ее соседей по ребрам с весами 0 или 1. Если вес ребра = 0, кладем вершину в начало очереди (`appendleft`), если = 1, то кладем в конец очереди (`append`). Что мы получаем на первом шаге? Сначала идут только вершины на расстоянии 0, а только затем - на расстоянии 1. Наша deque отсортирована. И сортировка сохраняется а каждом шаге.

Остается доказать, что, когда мы достаем вершину из очереди, для нее уже вычислено наименьшее расстояние от `start`.

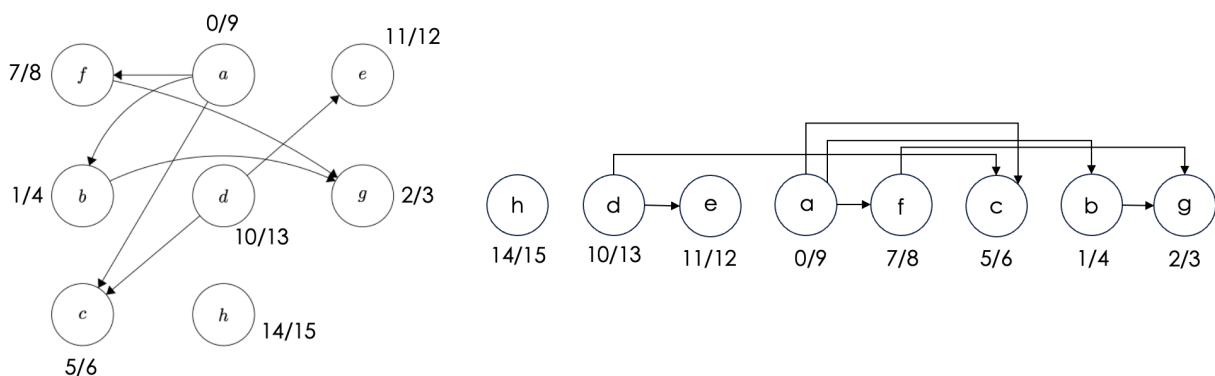
Пусть мы вытащили вершину v из начала очереди. Она имеет наименьшее расстояние $d_{curr}(s, v)$ от s из всех, находящихся в очереди (за ней могут быть вершины с такой же длиной пути, но все еще минимальное в очереди, см выше). Тогда предположим, что существует вершина u с расстоянием $d(s, u)$ такая, что $d_{curr}(s, v) > d(s, u) + d(u, v)$. Мы знаем, что $d(u, v) \in \{0, 1\}$. Но мы знаем, что вершина u лежит в очереди после вершины v , значит, $d(s, u) \geq d_{curr}(s, v)$. Но тогда мы получаем противоречие, а значит, $d(s, v)$ является минимальным. Тогда, когда мы достаем вершину из очереди, для нее уже вычислено минимальное расстояние.

Тогда перейдем к асимптотике. Каждое ребро релаксируется не более 1 раза, поэтому каждая вершина заносится в очередь не более раза. То есть мы по одному разу достаем вершину из очереди, для каждой вершины смотрим все ее ребра (всего для всех вершин $|E|$). Релаксация ребра и занесение в очередь производится за $O(1)$. Тогда общая асимптотика, как и требовалось, $O(|V| + |E|)$

Задание 3

Изучите топологическую сортировку (она делается на основе обхода в глубину). Рекомендуется использовать учебники Дасгупты и Кормена. Сделайте топологическую сортировку графа

DFS был проведен в алфавитном порядке



(a) Toposort (inspired by Kormen)

Задание 4

В графе может быть несколько кратчайших путей между какими-то вершинами. Постройте линейный по времени алгоритм, находящий количество вершин, которые лежат хотя бы на одном кратчайшем пути из s в t в неориентированном графе с единичными весами на рёбрах.

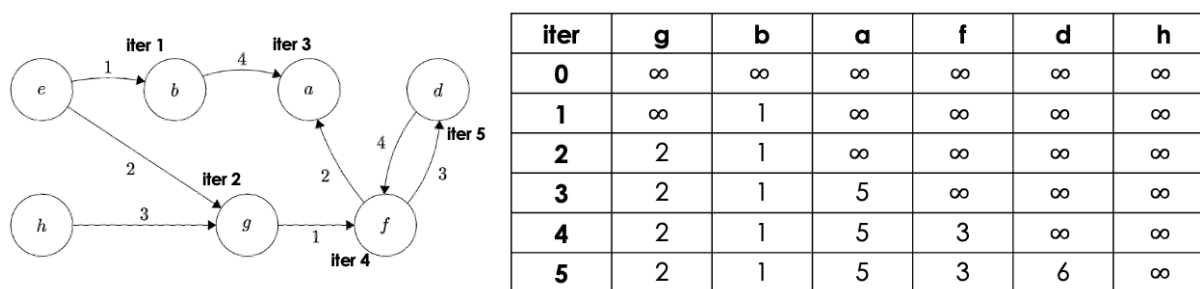
Так как весь ребер единичный, то мы можем считать длину пути просто за число шагов в BFS. Если бы мы шли за один BFS, то пришлось бы для каждой вершины хранить массив вершин, что не хочется. Так как нам в принципе не важно расположение путей, то заметим следующее: $d(s, v) + s(v, t) = d(s, t)$.

Проведем BFS с поиском кратчайшего пути (как только мы впервые дошли до вершины – нашли наименьший путь). Начинаем из s , для каждой вершины найдем $d(s, v)$, а также вычислим $d(s, t)$. Затем второй BFS проведем с началом в t , получим $d(t, v)$. Так как граф неориентированный, то $d(t, v) = d(v, t)$. Оба BFS работают за $O(|E| + |V|)$. Затем мы можем для каждой вершины проверить факт $d(s, v) + s(v, t) = d(s, t)$, ведь мы за два BFS посчитали все три расстояния из выражения. Если для k -л вершины выполняется равенство, то она лежит на каком-то минимальном пути. Такая проверка занимает $O(|V|)$, Total $O(|E| + |V|)$

Задание 5

Примените алгоритм Дейкстры к графу для поиска кратчайших путей от вершины e до всех остальных

Сделано по алгоритму Дейкстры, но пишу не все шаги, а только те, которые в итоге приводят к наименьшим путям для вершин (соответствующие ребра подписаны iter N)

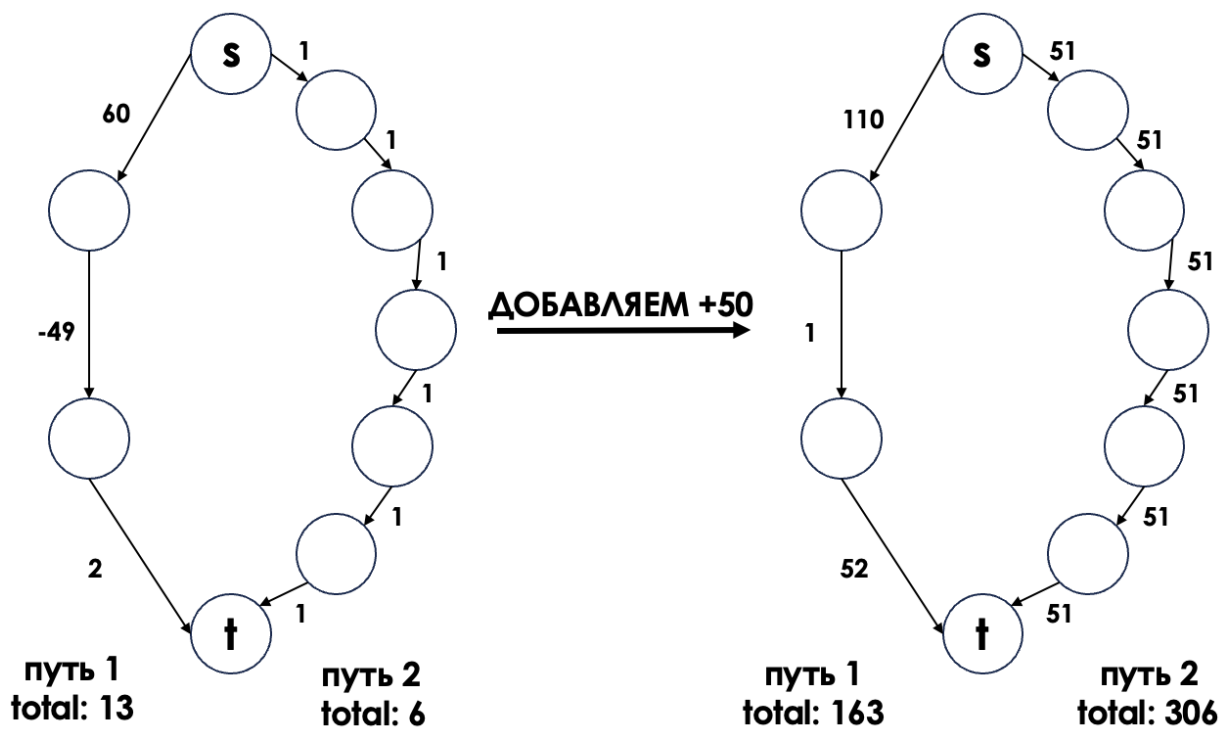


(a) Dijkstra's algorithm

Задание 6

Профессор О. П. Рометчивый предлагает следующий способ нахождения кратчайшего пути из s в t в ориентированном графе, содержащем рёбра отрицательного веса. Прибавим достаточно большую константу к весам всех рёбер и сделаем все веса положительными, после чего воспользуемся алгоритмом Дейкстры. Корректен ли такой подход? Если да, то докажете это, если нет — приведите контрпример.

Это просто неверно. Типа да, такая ситуация спасает отрицательные ребра, но граф, который мы получаем, совсем другой. Но в случаях, когда мы добавляем очень большую константу C , которая, условно, в 100 раз больше максимального элемента (и перебрасывает минимальный в положительные числа при добавлении), то что мы можем сказать? Пусть у нас был какой-то путь веса $w_1 + w_2 + w_3 \dots + w_k$. Прибавив к каждому ребру константу, мы получим, что новый вес пути $w_1 + w_2 + w_3 \dots + w_k + k * C$. Если C достаточно большая, то w_i могут выглядеть не более чем погрешность на ее фоне. Но тогда мы получим, что вес пути прямо пропорционален его длине, что в корне неверно, смотрим контрпример.



Этот рисунок довольно тривиален. Мы видим, что в реальности (левый рисунок) самый короткий путь - путь2 длины 6. Но прибавив даже минимальную константу, которая передвлет отрицательное ребро в положительное, мы получаем граф на рисунке 2 - в нем минимальным становится путь2. Это в принципе неверно